



MADAN BHANDARI MEMORIAL COLLEGE

(Affiliated to Tribhuvan University)

Anamnagar, Kathmandu



A MAJOR PROJECT REPORT ON

BHABISHYABOT – AI Guide for Your Career

Submitted by:

AYUSHMI ADHIKARI [29136/078]

SALINI JHA [29157/078]

SUDIP PRADHAN [29170/078]

Submitted to:

Department of Computer Science and Information Technology

November, 2025

SUPERVISOR RECOMMENDATION

This is to certify that the project report titled “BhabishyaBot: An AI-Driven Predictive Support System”, submitted by Ayushmi Adhikari, Salini Jha, Sudip Pradhan, in partial fulfilment of the requirements for the degree of B.Sc.in Computer Science and Information Technology, is the original work carried out under my supervision.

I recommend this project for evaluation as it meets the prescribed requirements and academic standards of Madan Bhandari Memorial College.

.....

Mr. Phul Babu Jha

Project Supervisor

Department of Computer Science

Madan Bhandari Memorial College

ACKNOWLEDGEMENT

The successful completion of the **BhabishyaBot** project has been possible through the guidance, support, and encouragement of many individuals. We express our sincere appreciation to everyone who directly or indirectly contributed to the development of this system.

We would like to extend our heartfelt gratitude to Madan Bhandari Memorial College and the entire administrative team for providing a supportive learning environment and the necessary resources to transform our conceptual ideas into a working project. Their continuous encouragement has played a vital role in shaping this work.

We are equally thankful to our supervisor, **Mr. Phul Babu Jha**, for his valuable insights, timely suggestions, and constant support throughout the research and development process. His mentorship helped us navigate challenges and significantly enhance the overall quality of this project.

With respect,

Ayushmi Adhikari

Salini Jha

Sudip Pradhan

ABSTRACT

BhabishyaBot is an AI-driven career guidance system designed to provide personalized career recommendations for students by analyzing their interests, motivations, academic background, and written responses. The system integrates Natural Language Processing (NLP) techniques to evaluate user essays for sentiment, intent, and key themes, while machine learning classification models determine suitable career paths with improved accuracy. The architecture incorporates structured data collection forms, a sentiment-aware essay analyzer, and a classification engine that maps user profiles to potential careers. Additionally, the system generates tailored career roadmaps highlighting essential skills, courses, and development steps. This report presents the overall design, workflow, and system architecture of BhabishyaBot, along with supporting diagrams such as DFDs, UML models, and database structures. The aim is to create an intelligent, scalable, and user-friendly platform that assists students in making informed career decisions.

TABLE OF CONTENTS

ACKNOWLEDGEMENT	iv
ABSTRACT	v
List of Abbreviation	viii
List of Figures	ix
List of Tables	x
1. Introduction	1
1.1 Introduction	1
1.2 Problem Statement	1
1.3 Objectives	2
1.4 Scope and Limitations	2
1.5 Development Methodology	3
1.6 Report Organization	3
2. Background Study and Literature Review	5
2.1 Background Study.....	5
2.2 Literature Review	7
3. System Analysis	8
3.1 Requirement Analysis.....	8
1. Functional Requirements	8
2. Non-functional Requirements	9
3.2 Feasibility Analysis	10
1. Technical Feasibility	10
2. Operational Feasibility	10
3. Economic Feasibility	10
4. Schedule Feasibility	11
3.3 Structured Analysis.....	12
1. Flowchart	12
2. DFD Diagram	13

4	System Design	15
4.1	Design	15
1.	System Architecture	15
2.	Database Design	16
3.	Form Design	17
4.	Interface Design	19
4.2	Algorithm Details.....	21
5	Implementation and Testing	27
5.1	Implementation.....	27
1.	Tools Used.....	27
2.	Implementations Details	28
5.2	Testing	32
1.	Test Case for Unit Testing	32
2.	System Testing	34
6	Conclusion and Future Recommendations	35
	References.....	36
	Appendices	37

LIST OF ABBREVIATION

AI	Artificial Intelligence
API	Application Programming Interface
CS	Computer Science
DL	Deep Learning
KNN	K- Nearest Neighbor Algorithm
LLM	Large Language Model
ML	Machine Learning
NLP	Natural Language Processing
NN	Neural Network
SVM	Support Vector Machine
SWE	Software Engineer

LIST OF FIGURES

Fig 1: Use Case Diagram	9
Fig 2: Flowchart	12
Fig 3: DFD Level 0 Diagram	13
Fig 4: DFD Level 1 Diagram	13
Fig 5: DFD Level 2 Diagram	14
Fig 6: System Architecture	16
Fig 7: Sequence Diagram for Form	18
Fig 8: Landing Page Interface	19
Fig 9: Chatbot Interface	19
Fig 10: Essay Section Interface	20
Fig 11: Career Recommendation Interface	20
Fig 12: Landing Page	38
Fig 13: Chatbot Page	38
Fig 14: Essay Page	39
Fig 15: Career Recommendation Page1	39
Fig 16: Career Recommendation Page2	40
Fig 17: Roadmap Page	40
Fig 18: College and Job Listing Page	41

LIST OF TABLES

Table 1: Schedule Feasibility.....	11
Table 2: User Table	17
Table 3: College Data Table	17
Table 4: Functional Requirement Implementation	29
Table 4: Database Unit Test Cases.....	33
Table 5: Chatbot Flow Test Cases.....	33
Table 6: Backend API Unit Test Cases.....	34

Chapter 1: INTRODUCTION

1.1 Introduction

In today's fast-changing world and an increasingly uncertain job market, students and job seekers in Nepal often struggle due to limited access to market knowledge and a lack of personalized guidance, as existing services are usually generic and not tailored to individual needs. BhabishyaBot addresses this gap as a web-based platform designed to help users explore suitable career options by considering their education, personality, and personal motivations. Through features such as career path recommendations using a Random Forest classifier, sentiment analysis of user-written essays to understand motivation and alignment, and job or college suggestions including available scholarships that are relevant to the user's chosen path, the system provides a personalized and interactive experience. With its practical applications in schools and colleges as a virtual counsellor and its potential deployment in local government or community centres as a digital help desk, BhabishyaBot can guide individuals in selecting appropriate careers and exploring the academic or professional routes best suited to them.

1.2 Problem Statement

Most existing career guidance tools today relies on generic aptitude tests or academic performance failing to understand individual's actual interest and motivations. These tools don't make any personalized reading for their users but rather use general Q/A to categorize users and suggest generic career title for their users.

Sakir Hossain Faruque et al. proposed a system that integrates NLP with ML and DL algorithm to suggests early careers path with precision for CS and SWE students. The system demonstrated reliable results under given questionnaires. However, it did not account for students from other faculty except CS and SWE students[1].

Naveen Mathews Renji et al. proposed a system that integrates LLMs to conduct mock career interviews and recommend next step based on resumes. The system enhances user experience by giving personalized career path analysis and recommending suitable courses from the popular platforms. Although insightful, their system lacks to provide roadmap and available courses in colleges and available scholarship on a local level [2].

Harshita et al. developed a system career buddy that uses NLP and NN to provide relevant guidance for their career quires. While the system provides relevant guidance, it lacks personalization which limits their accuracy [3].

These gaps highlight the need for a more adaptive, personalized and locally contextualized career guidance systema that can cater to a broader user beyond

2.1Objective

1. To develop a personalized career guidance platform that uses ML algorithm and NLP to analyze user inputs- user interest, motivations, educational background and essays to suggest suitable career paths.
2. To generate tailored career roadmaps for students, including relevant skills, course structures and progression plans based on their selected career path.
3. To provide job seekers with real-time job listings by integrating APIs from platforms like Mero job or Jobs Nepal, based on their profile and interest analysis.

1.3 Scope and Limitations

The scope of BhabishyaBot includes providing personalized career guidance by analysing user education, interests, motivations, and personality traits. The system offers career path recommendations using machine learning models, sentiment analysis of user-written essays, and localized suggestions for colleges, job opportunities, and available scholarships. It is designed for use in educational institutions, community centres, and individual career planning, serving students from diverse academic backgrounds. However, the system has certain limitations. The accuracy of recommendations depends on the quality and diversity of the dataset used for model training, which may not fully represent all Nepali demographic and socio-economic contexts. The platform also relies heavily on user inputs, meaning misleading or incomplete information can affect outcomes. Additionally, while BhabishyaBot provides general guidance and roadmaps, it does not replace professional counselling for highly specialized career domains, and its coverage of local opportunities is constrained by the availability of publicly accessible data.

1.4 Development Methodology

BhabishyaBot is developed using an Incremental Development Methodology, which allows the system to be built, tested, and refined in multiple manageable phases. This approach is suitable because the project integrates machine learning models, NLP components, and a web-based interface—each of which can be developed and improved progressively.

In the first increment, core functionalities such as user data collection, basic interface setup using FastAPI, and initial career recommendation using the Random Forest classifier were implemented. The second increment focused on integrating sentiment analysis for user essays, enhancing the recommendation engine, and improving UI interaction. The third increment added localized job and college suggestions, scholarship filtering, and dashboard enhancements. Each increment went through requirement validation, implementation, testing, and user feedback collection, enabling continuous refinement. This iterative approach ensured higher flexibility, minimized risks, and allowed new features to be added without disrupting previously developed modules.

1.5 Report Organization

This report is structured into six chapters to present the development of BhabishyaBot in a clear and systematic manner.

Chapter 1 introduces the project by outlining the background, problem statement, objectives, scope and limitations, the incremental development methodology adopted, and the overall structure of the report.

Chapter 2 provides the background study and literature review, where fundamental theories, key concepts, and terminologies related to machine learning, NLP, and career guidance systems are discussed, followed by a detailed review of existing research works and similar systems developed by other scholars.

Chapter 3 focuses on the system analysis, beginning with requirement analysis that includes both functional and non-functional requirements, illustrated where necessary using use case diagrams and descriptions. This chapter also presents feasibility analysis from technical, operational, economic, and schedule perspectives. Depending on the modelling approach used, the chapter further includes structured analysis with ER

diagrams and DFDs or object-oriented analysis incorporating class, state, sequence, and activity diagrams.

Chapter 4 details the system design, explaining the overall architecture, data design, transformation of ER diagrams into relational schemas, object or process modelling, component and deployment diagrams, along with the algorithmic logic behind the machine learning and NLP components used in the system.

Chapter 5 covers the implementation and testing of BhabishyaBot. It describes the tools, technologies, and platforms used, followed by module-wise implementation details including functions, methods, and algorithm integration. The chapter also presents the testing procedures, including unit and system testing, along with result analysis that evaluates system performance, model accuracy, and user interaction.

Chapter 6 concludes the report by summarizing the key findings, highlighting the significance of BhabishyaBot as a personalized career guidance platform, and offering future recommendations such as expanding datasets, integrating more advanced AI models, and improving local data coverage for enhanced career suggestions.

Chapter 2: BACKGROUND STUDY AND LITERATURE REVIEW

2.1 Background Study

Career guidance systems have evolved significantly with the advancement of artificial intelligence, natural language processing (NLP), and machine learning (ML). Traditionally, human experts assessed a student's interests, academic background, and personality traits to suggest suitable career paths. With the rise of digital transformation, these processes are being automated to deliver faster, data-driven, and personalized recommendations. BhabishyaBot is integrating core concepts from chatbot frameworks, NLP pipelines, classification algorithms, and database-driven user profiling.

1. Chatbot Systems and Conversational Interfaces

A chatbot is an AI program capable of having a conversation using text. Modern chatbot systems use rule-based logic or machine learning to understand user and generate meaningful responses. It uses a sequential conversational flow, which ensures structured data collection such as Name, Age and Essays. FastAPI serves as the backend engine that handles conversational states, user sessions, and API communication.

2. Natural Language Processing (NLP)

NLP enables machines to analyse human language. It includes processes such as tokenization, lemmatization, part-of-speech tagging, and keyword extraction. For BhabishyaBot:

- spaCy is used for linguistic preprocessing.
- TextBlob sentiment analysers determine the emotional tone of the user's essay.
- Keyword extraction techniques help identify thematic interests

These NLP techniques allow the system to convert unstructured text into structured features that guide the recommendation model.

3. Machine Learning for Classification

Career recommendation engines often rely on supervised machine learning models that map user attributes to predefined career clusters. BhabishyaBot uses RandomForest Classifier from scikit-learn, which is an ensemble learning algorithm that combines multiple decision trees to improve prediction accuracy. It is well-suited for:

- Handling mixed numerical and categorical features

- Providing robust predictions with minimal overfitting
- Ranking feature importance, which helps determine influential factors in career outcomes

The ML model takes user education, hobbies, essay-derived keywords, and sentiment values as input features.

4. Sentiment Analysis and Interest Profiling

Sentiment analysis identifies whether a user's writing expresses positive, neutral, or negative emotion. This information is relevant because motivation and emotional tone offer indirect cues about the user's passion toward certain fields. Interest profiling helps match user inclinations with suitable domains such as engineering, business, design, or social sciences.

5. Database Systems and User Profiling

Database systems store structured user information for future analysis and system improvement. BhabishyaBot uses Microsoft SQL Server to store the following data:

- Personal details
- Hobbies
- Education level
- Submitted essay

Relational databases ensure data consistency, reliability, and long-term availability, enabling the system to track user behavior and refine the ML model over time.

6. Web Technologies and System Integration

BhabishyaBot integrates multiple technologies into a cohesive platform:

- Frontend: React.js + Tailwind CSS for a responsive and clean UI
- Backend: FastAPI for high-speed API responses
- NLP/ML Modules: Python, spaCy, TextBlob/BERT, scikit-learn
- Database: SQL Server Management Studio

This combination allows real-time interaction, efficient text processing, and seamless data flow between frontend, backend, and database components.

2.2 Literature Review

In recent years, the use of AI and NLP has significantly improved how career development and coaching systems work. One key area is personalized learning and guidance, which helps people grow in their careers and also benefits organizations by improving employee skills and readiness [4].

Shilaskar et al. (2024) introduced a dual-layer system that uses machine learning to predict careers and a RASA-based chatbot to give students personalized advice. This system focuses on delivering customized guidance through chat, similar to the approach used in BhabishyaBot [5].

Casuat (2020) took a different approach by addressing the gap between what universities teach and what the job market needs. Their system used data from 3000 entries to predict employability using machine learning algorithms like SVM, Decision Trees, Naïve Bayes, and KNN. Among these, SVM gave the highest accuracy (92.22%) [6].

Ofoegbu et al. (2023) proposed a machine learning-based career guidance system using Decision Tree, Naïve Bayes, and Random Forest algorithms. Their model predicted suitable career paths for students based on academic performance, with Random Forest and Naïve Bayes showing the best accuracy. While effective in prediction, the system lacked interactive elements like chat-based guidance or essay analysis [7].

Probst et al. (2018) conducted an in-depth study on tuning strategies and the impact of hyperparameters in Random Forest algorithms. They analyzed how factors like the number of trees, node size, and feature sampling influence performance and accuracy. Their findings highlight that careful tuning significantly improves results, especially in classification tasks. This research supports BhabishyaBot's use of Random Forest by guiding how to optimize the model for accurate career recommendations [8].

Chapter 3: SYSTEM ANALYSIS

3.1 Requirement Analysis

3.1.1 Functional Requirements

1. Chatbot Interface:

The system must provide an intuitive chatbot interface to collect user information such as name, age, academic background and career preferences through conversations.

2. Essay Input Module:

Users will be prompted to write a short essay. The system will analyze sentiment and extract meaningful features from the text.

3. Sentiment Analysis:

It is implemented using Textblob and Spacy, this module will extract user inclination, motivation and confidence towards career goals to store personalized data about users.

4. Career Prediction Engine:

A trained Random Forest Classifier will use the user's demographic data and textual analysis results to recommend suitable career paths to the users.

5. College and Job Listing Interface:

Based on the chosen career, the system will list relevant academic institutions and real-time job listings using integration with Kantipur job or JobsNepal, based on the predicted career area for those who are job seeking.

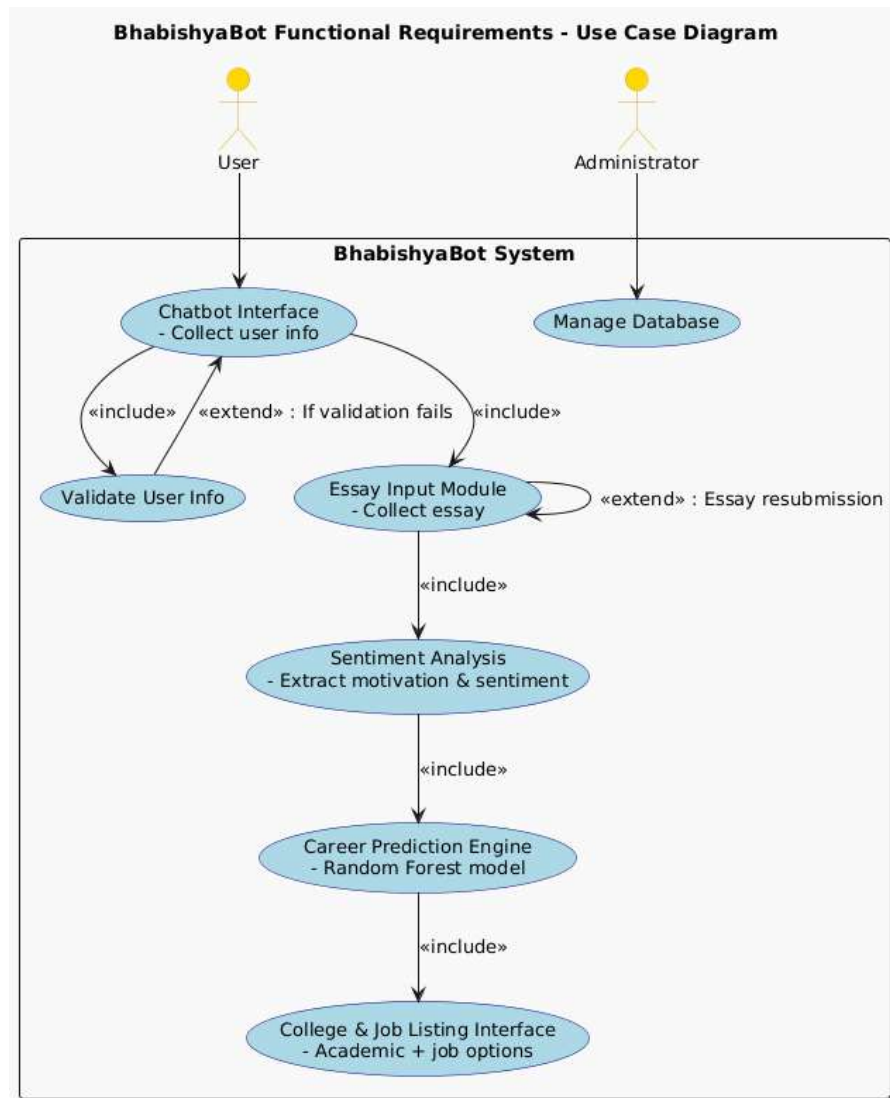


Fig 1: Use Case Diagram of Functional Requirements

3.1.2 Non-Functional Requirements

1. Usability:

The interface must be simple and responsive to cater to both tech-savvy users and those with minimal digital literacy.

2. Accuracy:

The machine learning model should maintain a reliable accuracy rate in predicting appropriate career paths.

3. Scalability:

The architecture must support potential future integration with external platforms, mobile apps, and multilingual datasets.

4. **Security and Privacy:**

Sensitive user data must be handled securely and anonymously. The system should comply with basic data privacy guidelines.

5. **Portability:**

The application should be deployable on standard web servers and accessible across major browsers and devices.

3.2 Feasibility Study

The feasibility study evaluates the practicality of developing and deploying the AI-powered Career Guidance Web Application within the given time and resource constraints.

1. Economic Feasibility

All the libraries and frameworks used in the system are free and open-source, which eliminates software licensing costs. The development environment requires only a personal computer with internet access, and the system can be hosted on free-tier platforms (e.g., Render, GitHub Pages, or Replit). This makes the project highly cost-effective and feasible within the constraints of an academic project timeline and budget.

2. Technical Feasibility

The system is technically feasible using widely adopted, well-documented technologies such as Python, FastAPI, TextBlob, spaCy, and scikit-learn. All major components (frontend, backend, ML models, NLP processing, and dataset handling) are achievable using open-source tools. The integration of REST APIs for job listings and dataset-based college suggestions also ensures the system is modular, scalable, and easy to maintain. No proprietary software or high-end infrastructure is required, making it suitable for academic and small-scale deployment.

3. Operational Feasibility

BhabishyaBot is designed to operate as a user-friendly web-based platform that does not require installation or technical skills from the user. Users can interact with the chatbot and receive meaningful career guidance within minutes. The backend can be run on any standard local or cloud server with basic configuration. Since the platform uses a conversational interface and automated recommendations, there is minimal need for manual supervision or support during usage.

4. Schedule Feasibility

S.N	Task	Estimated Duration	Dependencies	Scheduled Timeline
1	Requirement Analysis & Background Study	2 weeks	None	July (Week 1–2)
2	Functional & Non-Functional Requirements + Use Case Modelling	1 week	Task 1	July (Week 3)
3	System Design (Architecture, DFD/Class Diagrams, DB Design)	2 weeks	Task 2	July (Week-4) – Aug (Week 1)
4	Frontend Development (FastAPI Templates / UI)	3 weeks	Task 3	Aug (Week 2–4)
5	Backend Development (APIs + Core Logic)	3 weeks	Task 3	Aug (Week 2) – Sept (Week 1)
6	ML Module – Dataset Collection & Preprocessing	1.5 weeks	Task 3	Sept (Week 1–2)
7	ML Model Training, Optimization, Evaluation	2 weeks	Task 6	Sept (Week 3–4)
8	ML Integration with Backend	1.5 weeks	Tasks 5 & 7	Early October (Week 1–2)
9	System Testing (Unit, Integration, UAT)	2 weeks	Task 8	October (Week 2–3)
10	Deployment & Documentation	1.5 weeks	Task 9	October (Week 3–4)

Table 1: Schedule Feasibility

The schedule feasibility of the BhabishyaBot project was evaluated using a detailed schedule feasibility table outlining each task, its duration, dependencies, and planned timeline from July to October. Based on these estimates, the total required time is about 14 weeks, which fits within the available 16-week period. This provides a buffer for minor delays. Therefore, the project schedule is feasible, and the system can be completed within the timeframe.

3.3 Structured Analysis

1. Flowchart

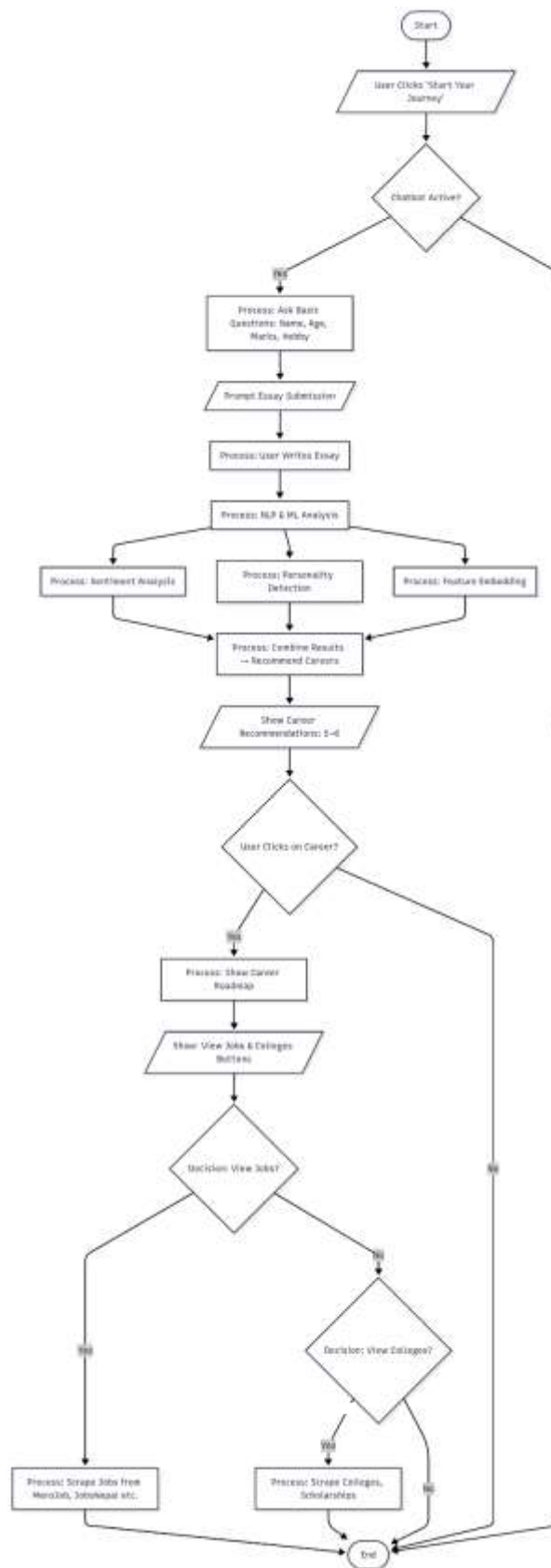


Fig 2: Flowchart of BhabisyBot

2. DFD Diagram

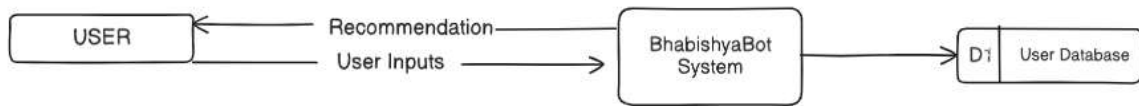


Fig 3: DFD Level 0 Diagram of BhabishyaBot

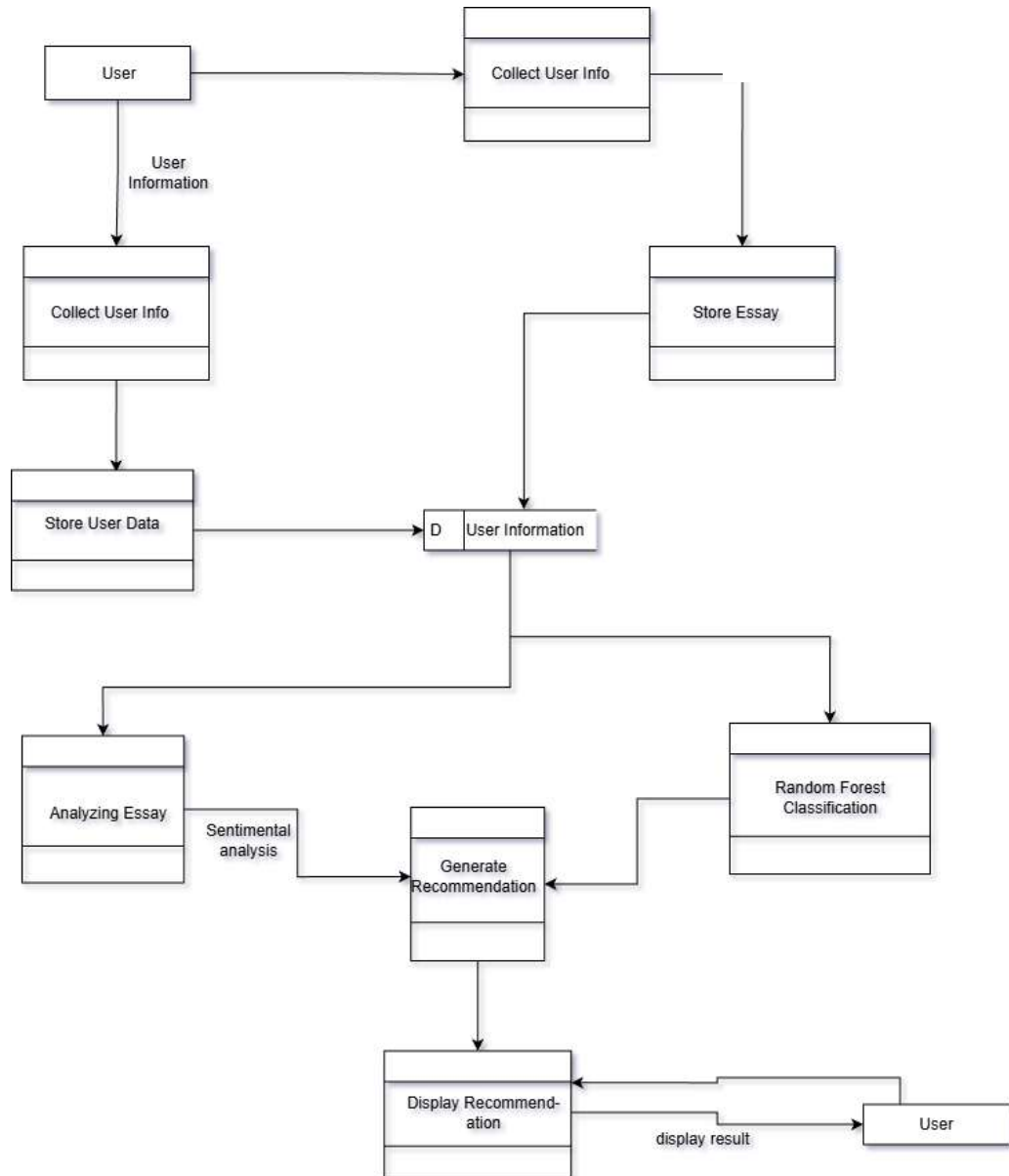


Fig: DFD Level 1 of Bhabisyabot

Fig 4: DFD Level 1 Diagram of BhabishyaBot

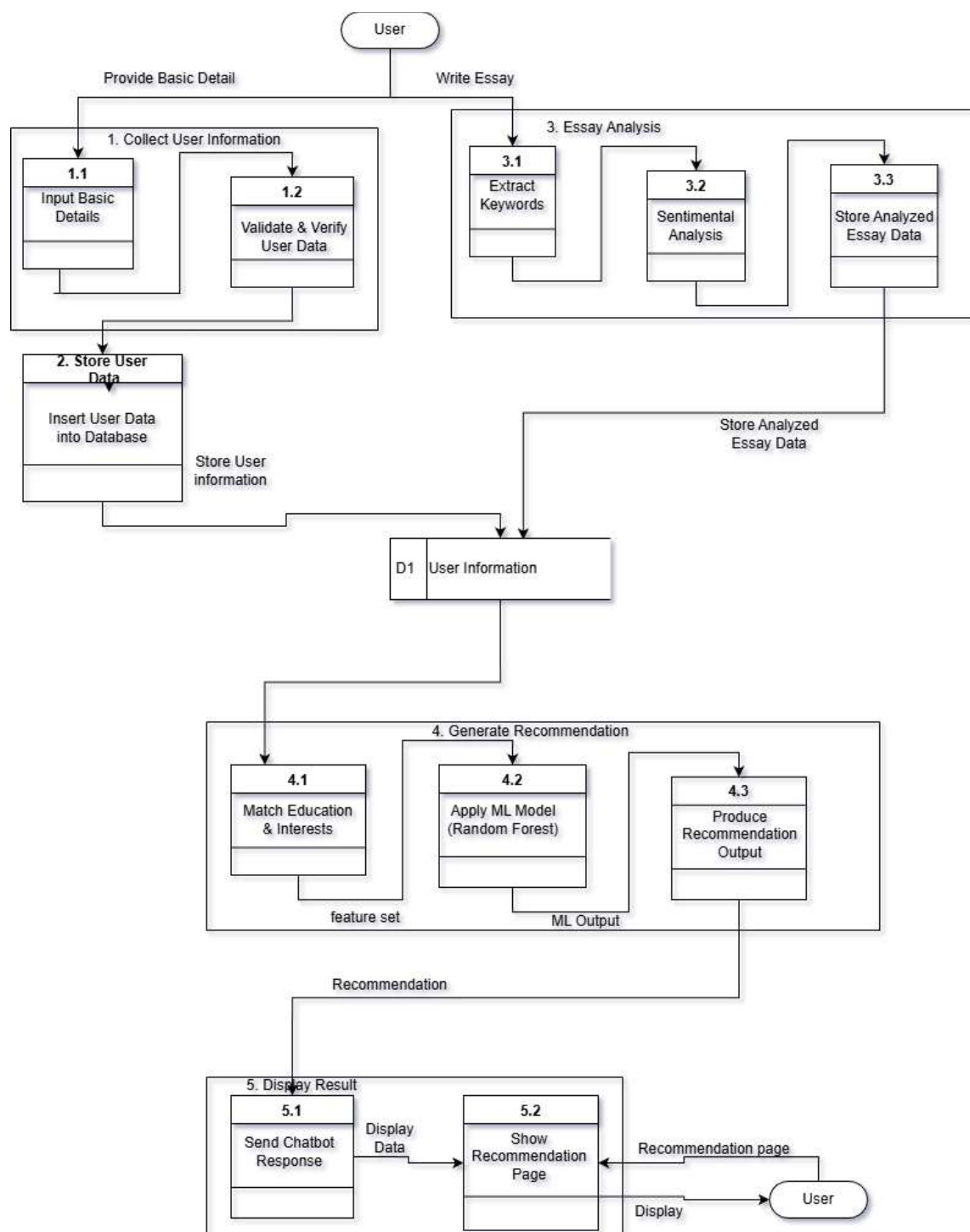


Fig: DFD Level 2 of Bhabisyabot

Fig 5: DFD Level 2 Diagram of BhabishyaBot

Chapter 4: SYSTEM DESIGN

4.1 Design

System design refers to the process of defining the architecture, components, modules, interfaces, and data for a software system. It involves creating a blueprint or plan for the system that specifies how it will be built, how it will function, and how it will meet the requirements of the clients.

1. System Architecture

The BhabishyaBot Career Guidance System follows a modular three-layer architecture with clear separation of concerns, ensuring scalability and smooth data flow across components.

i. **Presentation Layer:**

This layer collects user details—interests, background, and essay input—and displays chatbot replies, predicted careers, and analysis results. It communicates with the backend through REST API calls.

Key functions include:

- Web forms for data collection
- Display of recommendations and sentiment output
- API communication with FastAPI backend

ii. **Application Layer:**

- **NLP Module:** Extracts keywords, performs sentiment analysis, identifies intent, and prepares structured text features for ML.
- **Machine Learning Module:** Combines user profile + NLP features, runs classifiers (Random Forest, Logistic Regression, etc.), and predicts suitable careers with confidence scores.
- **Recommendation Engine:** Converts predictions into meaningful outputs by mapping skills, courses, and job trends into a structured career roadmap.

iii. **Data Layer:**

A simplified layer storing user-related data essential for analysis and output generation.

- **Users Table:** Stores profile info, essay sentiment, extracted keywords, and ML prediction.

Simplified System Architecture - BhabishyaBot

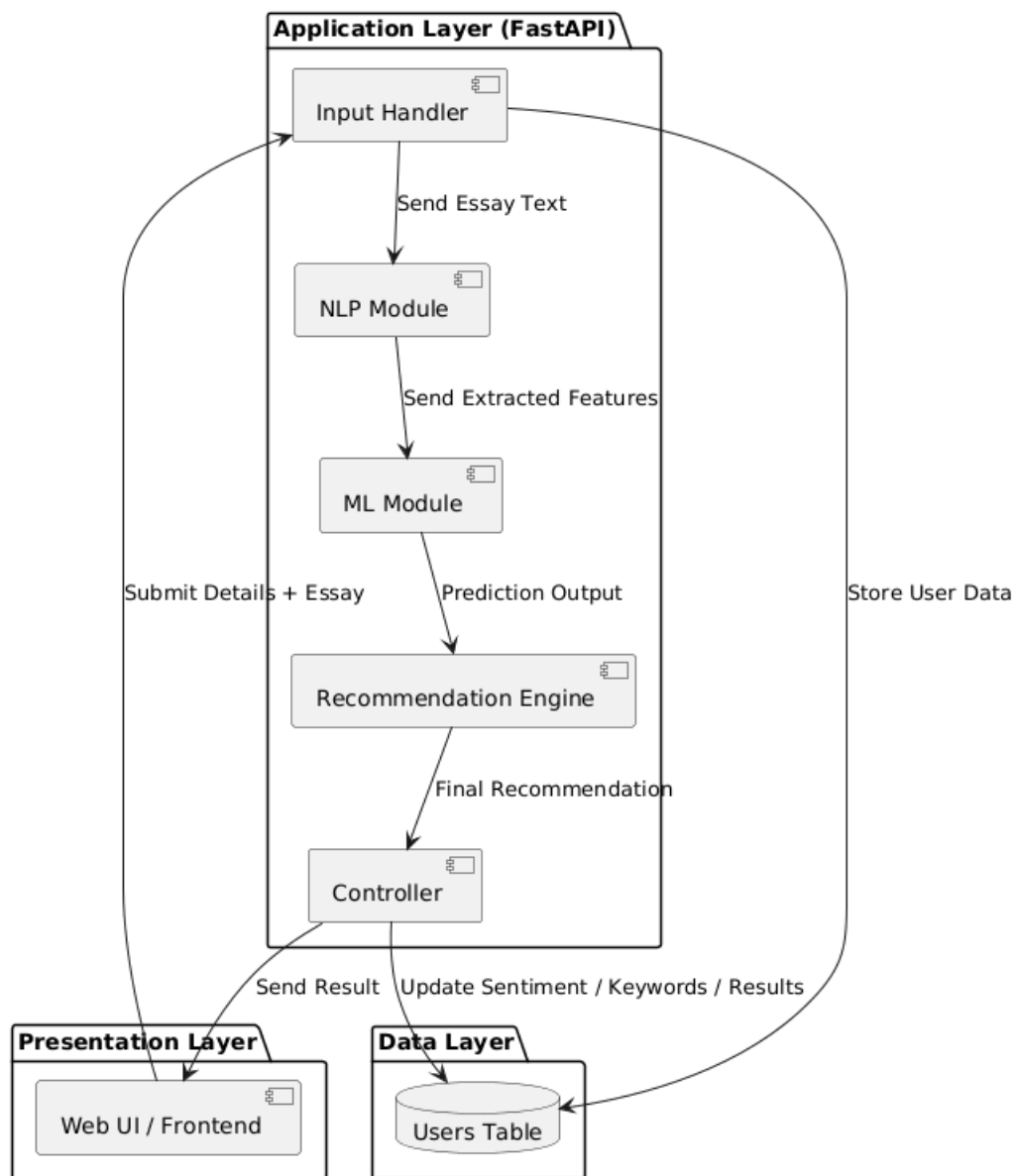


Fig 6: System Architecture

2. Database Design

- **User Table:** This table stores the data of users from chatbot interaction.

Table 4.1: User Table

Field Name	Data Type	Constraints	Description
User_id	INT	Primary Key, Auto Increment	Unique ID for each user
Name	NVARCHAR	NOT NULL	User's full name
Age	INT	NOT NULL	User's Age
Education	NVARCHAR	NOT NULL	User's educational qualification
Hobbies	NVARCHAR	NOT NULL	User's hobbies
Essay	TEXT	NOT NULL	User's essay for NLP analysis
EssaySentiment	INT		
EssayKeywords	NVARCHAR		

3. Forms Design

Forms are key in interactive systems, serving as the main interface for user input. In BhabishyaBot, they collect details from basic information to essays. The sequence diagram shows how the user, interface, and backend modules like NLP and the classifier interact, clarifying how inputs are processed into career recommendations.

areas, and navigation cues without focusing on detailed styling or content. These prototypes help visualize the user journey, ensure usability, and guide the development of a clean, structured, and intuitive interaction experience.

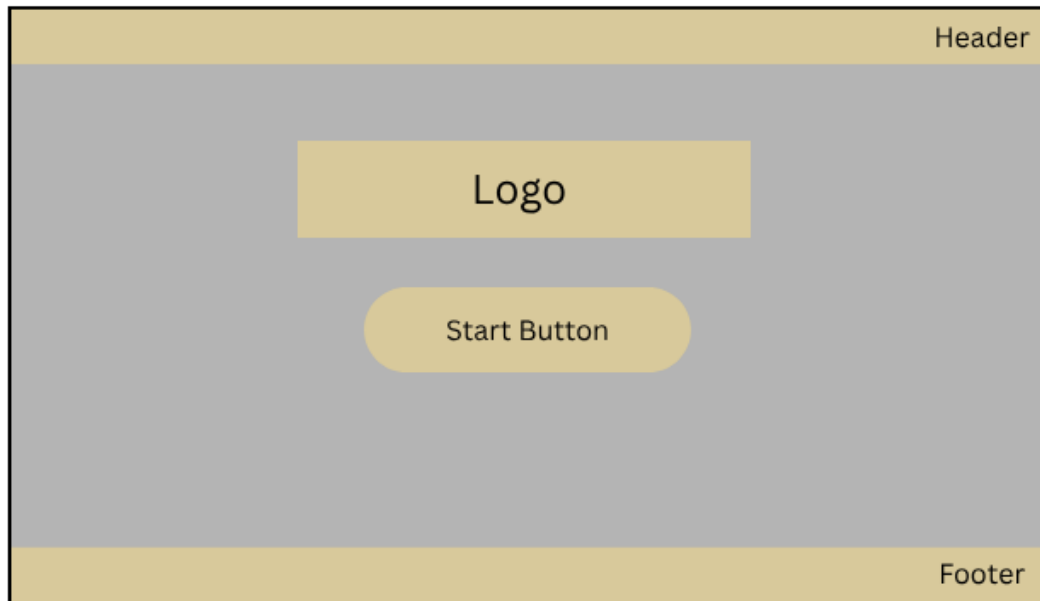


Fig 8: Landing Page Interface

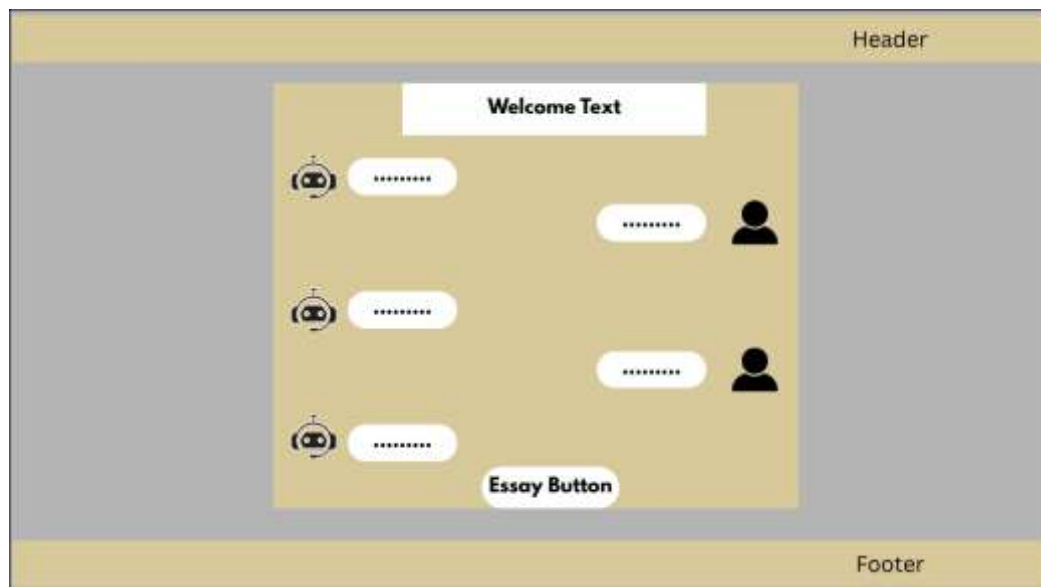


Fig 9: Chatbot Interface

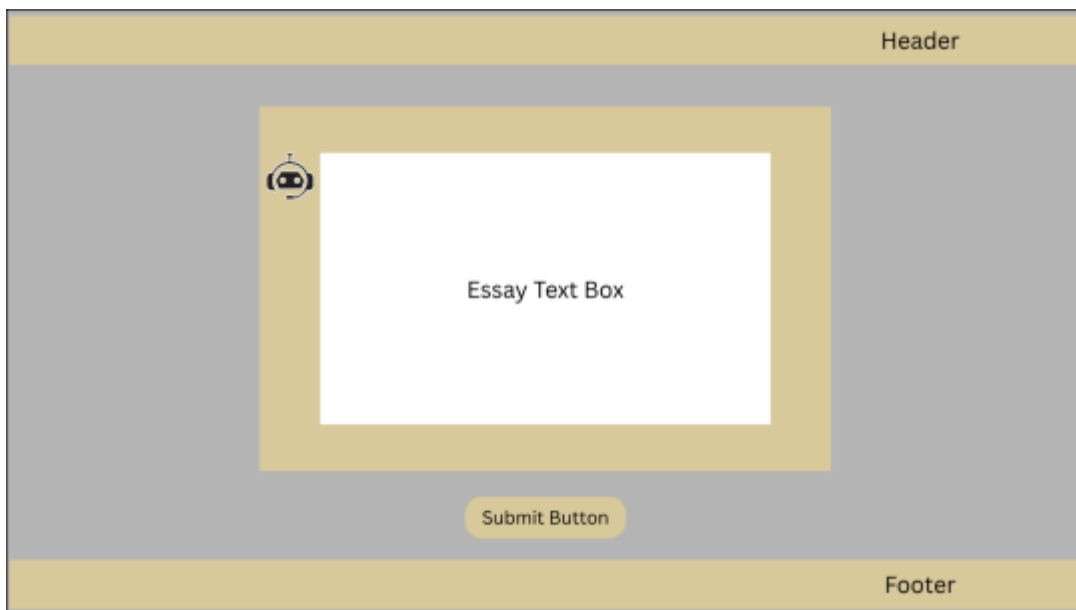


Fig 10: Essay Section Interface

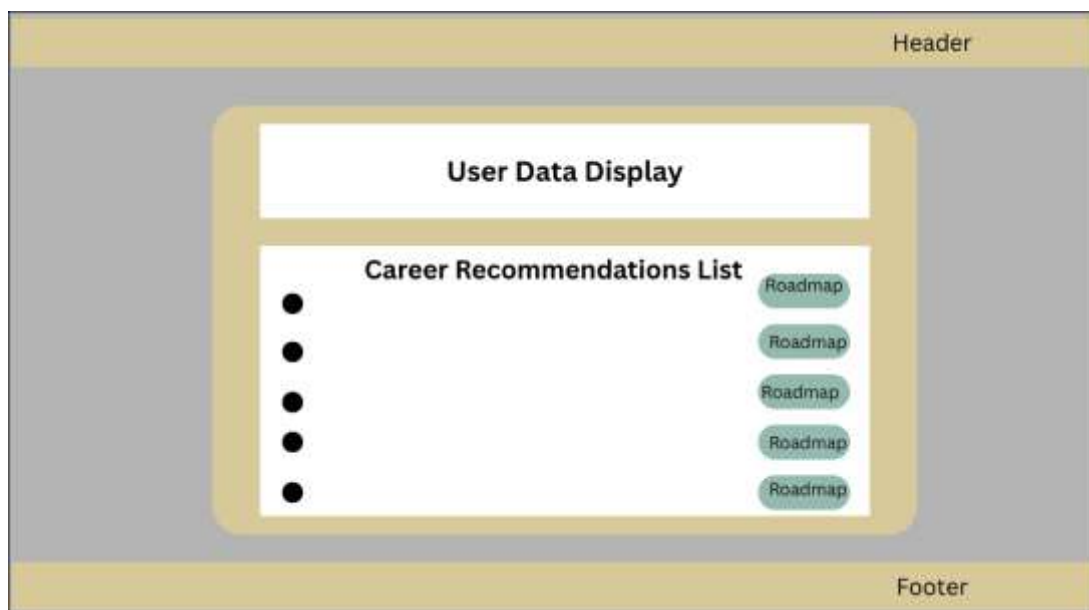


Fig 11: Career Recommendation Interface

4.2 Algorithm Details

1. NLP Algorithm

This algorithm converts raw user essays and hobby/education inputs into structured numerical features. These features act as the foundation for accurate machine-learning-based career predictions. The algorithm follows a hybrid NLP approach combining lexicon-based sentiment scoring, multi-hot encoding, and keyword detection.

Step 1: Text Preprocessing

Text preprocessing ensures that the essay is normalized and stable before further NLP operations. The essay text E is cleaned by:

- Converting to lowercase
- Removing unnecessary characters
- Handling empty or malformed input
- Ensuring compatibility with downstream models like TextBlob

Mathematically, preprocessing transforms:

$$E' = \text{normalize}(E)$$

Where *normalize()* performs case-folding and whitespace cleanup. This step improves the consistency of sentiment scoring and keyword detection.

Pseudocode:

```
PREPROCESS_TEXT(E):  
    E_clean ← lowercase(E)  
    E_clean ← remove_invalid_characters(E_clean)  
    return E_clean
```

Step 2: Sentiment Polarity Extraction

Sentiment analysis is performed using TextBlob's polarity function. This lexicon-based method assigns a polarity score:

$$S(E') \in [-1, 1]$$

- +1 → highly positive tone
- 0 → neutral tone
- -1 → highly negative tone

Sentiment represents the emotional attitude expressed in the essay and helps infer internal motivation or interest patterns.

A protective wrapper ensures the system continues working even if TextBlob fails (e.g., due to malformed text):

$$S_{safe}(E') = \begin{cases} S(E'), & \text{if successful} \\ 0, & \text{if error occurs} \end{cases}$$

Pseudocode:

```

EXTRACT_SENTIMENT(E_clean):
    try:
        sentiment ← TextBlob(E_clean).polarity
    except:
        sentiment ← 0.0
    return sentiment

```

Step 3: Multi-Hot Encoding of User Hobbies

Hobbies, expressed in natural language, are first split into lists. A global set of all hobby categories is derived from the dataset. For each hobby feature h_i in the hobby set $H = \{h_1, h_2, \dots, h_n\}$:

$$hobby_i = \begin{cases} 1, & \text{if } h_i \text{ appears in the user's hobby list} \\ 0, & \text{otherwise} \end{cases}$$

This converts variable-length hobby text into a fixed-size binary vector capturing personal interest patterns closely tied to career preferences.

Pseudocode:

```

ENCODE_HOBBIES(user_hobbies, ALL_HOBBIES):
    vector ← []
    for each hobby h in ALL_HOBBIES:
        if h in user_hobbies:
            vector.append(1)
        else:
            vector.append(0)
    return vector

```

Step 4: Keyword-Based Essay Encoding

A predefined list of career-related keywords $K = \{k_1, k_2, \dots, k_m\}$ is used to detect thematic alignment within the essay. Each keyword feature is encoded as:

$$kw_i = \begin{cases} 1, & \text{if keyword } k_i \subseteq E' \\ 0, & \text{otherwise} \end{cases}$$

These binary values indicate whether the user implicitly expresses interest in areas such as management, coding, research, art, finance, data, or sciences. This bridges semantic meaning and machine learning input.

Pseudocode:

```
ENCODE_KEYWORDS(E_clean, KEYWORDS):  
    vector ← []  
    for each keyword k in KEYWORDS:  
        if k occurs in E_clean:  
            vector.append(1)  
        else:  
            vector.append(0)  
    return vector
```

Step 5: Final Feature Vector Construction

After executing steps (a)–(d), all outputs are merged into a single numerical vector:

$$\mathbf{x} = [Age, Education_{enc}, Sentiment] + HobbyVector + KeywordVector$$

This vector becomes the input for the Random Forest classifier. It represents a combination of emotional tone, interests, background, and topic relevance, providing a strong foundation for accurate career prediction.

2. Random Forest Algorithm

A Random Forest is an ensemble machine learning algorithm composed of N independent decision trees, where:

- Each tree is trained on a bootstrap sample.
- Each split in each tree considers only a random subset of features, increasing diversity.
- The final output is generated through majority voting, reducing overfitting and improving accuracy.

Mathematically:

$$\hat{y} = \text{mode}(T_1(x), T_2(x), \dots, T_N(x))$$

Where:

- T_i = the i^{th} decision tree
- $\hat{y}$ = final predicted class

Step 1: Data Encoding

Dataset is as:

$$D = \{(X_1, y_1), (X_2, y_2), \dots, (X_m, y_m)\}$$

Categorical values (education, hobbies, etc.) are encoded:

$$X_{\text{encoded}} = f(X)$$

This makes all features numeric for the Random Forest.

Pseudocode:

```
INPUT: Raw dataset D
FOR each sample (Xi, yi) in D:
    Clean and normalize values
    Encode categorical features → LabelEncoder()
    Multi-hot encode hobbies
    Compute keyword flags
END FOR
```

Step 2: Bootstrap Sampling

For each tree T_i :

$$D_i = \text{bootstrap}(D)$$

Pseudocode:

```
FOR each tree i in N:
    Di = sample_with_replacement(D_train)
    Use Di as training set for tree i
END FOR
```

Step 3: Train–Test Splitting

To evaluate the algorithm's generalization ability, the dataset is split into:

- Training set (80%) → used to build the decision trees
- Testing set (20%) → used to evaluate accuracy

Random Forest benefits strongly from randomness, and a fixed `random_state` ensures reproducibility.

Mathematically, the split is:

$$(X_{train}, X_{test}, y_{train}, y_{test}) = \text{Split}(X, y, 0.2)$$

Pseudocode:

```
SPLIT_DATA(X, y):  
    X_train, X_test, y_train, y_test ← train_test_split(X, y, test_size=0.2)  
    return X_train, X_test, y_train, y_test
```

Step 4: Random Feature Selection

At each node of each tree:

$$F_i = \text{random subset of features from } X$$

Split chosen by minimizing **Gini impurity**:

$$Gini = 1 - \sum_{k=1}^K p_k^2$$

Pseudocode:

```
FOR each node in tree i:  
    Fi = randomly_select_features(X)  
    FOR each possible split using Fi:  
        Compute Gini impurity  
    END FOR  
    Choose split with lowest Gini  
END FOR
```

Step 5: Random Forest Model Training

Random Forest builds N independent decision trees.

Each tree receives:

- A bootstrap sample of the training data
- A random subset of features at each split

For each tree T_i :

$$T_i = \text{DecisionTree}(\text{bootstrap}(X_{train}, y_{train}))$$

The full forest is:

$$RF = \{T_1, T_2, \dots, T_N\}$$

The model then learns decision boundaries that map the input feature space to a career label.

Pseudocode:

```
TRAIN_RANDOM_FOREST(X_train, y_train):  
    model ← RandomForestClassifier(n_estimators=200)  
    model.fit(X_train, y_train)  
    return model
```

Step 6: Aggregated Prediction (Majority Voting)

Each tree predicts:

$$y_i = T_i(x)$$

Final prediction:

$$\hat{y} = \text{mode}(y_1, y_2, \dots, y_N)$$

Pseudocode:

```
INPUT: New sample x  
FOR each tree Ti in forest:  
    yi = Ti.predict(x)  
END FOR  
 $\hat{y}$  = majority_vote( y1, y2, ..., yN )  
RETURN  $\hat{y}$ 
```

Chapter 5: IMPLEMENTATION AND TESTING

5.1 Implementation

5.1.1 Tools Used

A. CASE Tools

1. Draw.io (Diagramming Tool)

Draw.io was used to design structured analysis diagrams such as Data Flow Diagrams (DFDs) and workflow representations. Its intuitive drag-and-drop interface, cloud integration, and compatibility with standard modelling notations made it suitable for visualizing system processes and data interactions.

2. GitHub (Version Control and Collaboration)

GitHub served as the central platform for version control, team collaboration, and code repository management. It facilitated features such as commit tracking, issue handling, branch management, and automated documentation.

3. Notion (Documentation and Project Management)

Notion was used for organizing documentation, writing meeting notes, creating to-do lists, and planning project milestones. It helped maintain structured documentation throughout the development phase, improving clarity and team coordination.

B. Programming Languages and Frameworks

1. Python (Backend and Machine Learning)

Python was the primary programming language for implementing the backend logic, chatbot workflow, machine learning model, and API endpoints. Libraries such as Scikit-Learn were used for model training and inference, while FastAPI provided a lightweight backend architecture.

2. React.js (Frontend Development)

React.js was used to develop the dynamic and interactive user interface of BhabishyaBot. Its component-based architecture enabled the creation of reusable UI components and ensured efficient rendering of recommendations and chat interactions.

3. Tailwind CSS (Styling Framework)

Tailwind CSS was used to style the frontend, enabling rapid UI development with utility-first CSS classes. It ensured consistent design, responsiveness, and flexibility in creating modern and clean user interfaces.

4. Scikit-Learn (Machine Learning Framework)

Scikit-Learn was the core machine learning library used for model training, evaluation, and deployment. It provided tools for data preprocessing, model selection, vectorization, and performance optimization.

C. Database Platforms

1. SQL Server (Backend Database)

SQL Server was employed as the primary database platform for storing user information, chat history, and other structured data required by the system. Its reliability, ACID compliance, and support for relational data modeling made it suitable for maintaining persistent records and ensuring secure data operations.

2. CSV Dataset (Machine Learning Input Data)

The machine learning module used a CSV dataset (`career_dataset.csv`) to train the recommendation model. This dataset was stored in the project's data directory and acted as the structured input for feature extraction and model training.

5.1.2 Implementation Details of Modules:

1. Requirement Mapping to Modules

The implementation of BhabishyaBot follows a structured mapping of system requirements into clearly defined modules.

Each **functional requirement** such as collecting user details, processing essays, predicting career paths, and generating personalized roadmaps, are assigned to a dedicated component within the system architecture.

Requirement	Module	Implementation Detail
Collect user information	User Management Module	FastAPI forms, validation using Pydantic
Analyze essay sentiment	Essay Analysis Module	NLP preprocessing + sentiment scoring
Extract keywords	Essay Analysis Module	RAKE/TF-IDF extraction
Predict career	ML Classification Module	Pretrained Random Forest/Logistic Regression
Generate career roadmap	Roadmap Module	Rule-based mapping, JSON data source

Fig: Functional Requirement Implementation

Non-functional requirements played an equally important role in shaping the implementation. Performance considerations led to lightweight API responses, efficient NLP preprocessing, and caching of trained ML models to minimize prediction latency. Scalability influenced the modular architecture, enabling independent updates of the NLP, ML, or roadmap modules without affecting others. Security requirements informed input validation, secure storage of user information, and controlled access to prediction APIs. Usability and accessibility were addressed through a clean UI design, meaningful feedback messages, and minimal steps for user interaction.

2. Module-wise Implementation

i. User Management Module

The User Management Module handles all interactions related to form submission, input validation, and database updates. When a user fills in their details and submits the form, FastAPI receives the data through a Pydantic model that validates fields such as interests, hobbies, motivations, and educational background. Once validated, the system stores the information in the database and prepares it for NLP processing.

```
@app.post("/submit-form")
async def submit_form(data: UserForm):
    user_id = save_user_to_db(data)
    return {"message": "Form submitted", "user_id": user_id}
```

ii. Essay Sentiment & Keyword Extraction Module

The Essay Sentiment & Keyword Extraction Module processes the user's written essay using a combination of text preprocessing, sentiment scoring, and keyword extraction techniques. The workflow begins with preprocessing, where the essay is converted to lowercase, tokenized, stripped of stopwords, and optionally lemmatized to normalize the text. This step improves the effectiveness of both sentiment analysis and keyword extraction by reducing noise.

For sentiment analysis, tools like TextBlob or VADER are used. The system computes a polarity score that indicates whether the essay expresses a positive, negative, or neutral tone.

```
from textblob import TextBlob  
  
def get_sentiment(text):  
    analysis = TextBlob(text)  
    return analysis.sentiment.polarity
```

Behind the scenes, TextBlob calculates sentiment using a lexicon-based approach where each word contributes a weight to the final sentiment score. A higher polarity may indicate confidence or enthusiasm, while a lower score might reflect uncertainty or stress both of which are important behavioural indicators for career prediction.

EssaySentiment and EssayKeywords for use in the prediction module. This module serves as the bridge between raw user input and structured analytical features.

iii. Career Prediction Module

The Career Prediction Module is responsible for generating a career recommendation using machine learning algorithms. The system loads a pre-trained classification model—such as Random Forest or Logistic Regression—which has been trained on a dataset that includes user interests, hobbies, motivations, sentiment, and keyword features.

The model is loaded at application startup using Joblib to avoid reloading it for every request:

```
import joblib
model = joblib.load("career_model.pkl")

During prediction, the module constructs a feature vector from the processed user
data and passes it to the model:

def predict_career(features):
prediction = model.predict([features])
return prediction[0]
```

Internally, Random Forest works by aggregating the outputs of multiple decision trees. Each tree evaluates the input features and votes for a career class, and the final prediction is determined by majority voting. This ensemble approach improves accuracy, reduces overfitting, and handles nonlinear relationships present in user data. The module returns a single predicted career label, which becomes the input for the roadmap generator.

iv. Career Roadmap Generator Module

The Career Roadmap Generator Module transforms the predicted career into a personalized learning and development path. This module is rule-based, relying on predefined datasets or JSON files that map each career to required skills, recommended courses, tools, and progression steps.

```
roadmaps = {
    "Software Developer": {
        "skills": ["Python", "DSA", "Databases"],
        "courses": ["CS50", "Python Bootcamp"],
        "steps": [
            "Learn basics of programming",
            "Build small projects",
            "Master DSA",
            "Create portfolio"
        ]
    }
}
```

}

When a prediction is made, the module retrieves the corresponding roadmap and formats it for frontend display. The flow ensures consistency: the ML output determines the roadmap, the roadmap directs the user toward skill-building, and the UI presents it in a structured, easy-to-follow manner.

3.Workflow Implementation

The overall workflow integrates all modules into a single automated pipeline. When the user submits their details and essay, the backend first validates the inputs and stores them. The essay is passed to the NLP module for sentiment and keyword extraction, and the results are combined with user interests and background to form a comprehensive feature set. This feature vector is then fed into the ML model, which predicts the career category. Finally, the roadmap module generates a career path that is sent back to the frontend for display. FastAPI endpoints ensure smooth data flow, low latency, and clear separation between processes.

4.UI/UX Implementation

The user interface focuses on clarity and simplicity, ensuring that users can navigate the system without confusion. The design includes clean forms with proper labels, inline validation for incorrect inputs, and clearly structured output pages that present the predicted career and roadmap. Responsive layouts allow the interface to adapt to different device sizes, while minimal styling ensures that users focus on the content rather than unnecessary visuals. The UI is built using FastAPI templates, allowing seamless integration between backend data and frontend presentation

5.2 Testing

5.2.1 Test Cases for Unit Testing

Unit testing was performed to verify the correctness of individual modules and functions used in the BhabishyaBot system. The tests focused on three major components:

(1) Database operations, (2) Chatbot flow control, and (3) Backend API responses. Each test case includes the objective, input, expected output, and actual result.

A. Unit Test Cases for Database Module (database.py)

Test Case ID	Test Objective	Input	Expected Output	Actual Output	Status
DB-01	Verify that a user record is inserted successfully into the Users table	Name="Test", Age=20, Education="BSc CSIT", Hobbies="Reading", Essay="Sample essay"	One new row is added to the Users table without errors	Row inserted successfully	Pass
DB-02	Ensure insertion fails safely when mandatory fields are missing	Missing Age	Operation should fail or return validation error without inserting incomplete row	Error handled without database change	Pass
DB-03	Test database connection initialization	Connect to SQL Server	Connection established without timeout	Connection successful	Pass
DB-04	Validate prevention of SQL injection	Input includes ' OR 1=1 -	Input should be sanitized and no malicious effect occurs	No injection; query executed safely	Pass

Table 2: Database Unit Test Cases

B. Unit Test Cases for Chatbot Flow (chatbot_logic.py)

Test Case ID	Test Objective	Input	Expected Output	Actual Output	Status
CB-01	Test greeting step	User starts conversation	Bot returns: "Hello! What is your name?"	Correct response returned	Pass
CB-02	Test name input transition	Name = "Salini"	Bot asks for age	Correct transition	Pass
CB-03	Test age validation	Age = "abc" (invalid)	Bot returns: "Please enter a valid age."	Validation handled correctly	Pass
CB-04	Test sequential flow completion	Name → Age → Education → Hobbies → Essay	Bot reaches essay prompt and waits for input	Sequential flow executed correctly	Pass
CB-05	Test re-prompt on empty response	Input=""	Bot asks user to re-enter the required information	Correct handling	Pass

Table 3: Chatbot Flow Test Cases

C. Unit Test Cases for Backend API (main.py)

Test Case ID	API Endpoint	Test Objective	Input	Expected Output	Actual Output	Status
API-01	/chat (POST)	Validate chatbot response API	JSON: {"message": "Hi"}	200 OK + JSON response containing next bot message	Correct response returned	Pass
API-02	/analyze-essay (POST)	Test essay analysis endpoint	JSON: {"essay": "I love coding."}	Keywords + sentiment values returned in JSON	Correct JSON structure	Pass
API-03	/recommend (POST)	Test recommendation endpoint	JSON with user details	List of recommended careers	Correct output	Pass
API-04	Any endpoint	Check server behavior for invalid payload	Missing required fields	422 Validation Error returned	Error returned as expected	Pass
API-05	Server health	Test if API server is reachable	GET request	200 OK	Server reachable	Pass

Table 4: Backend API Unit Test Cases

5.2.2 System Testing

System testing evaluated BhabishyaBot as a fully integrated application to ensure that all components functioned correctly as a single system. The complete workflow—from user input submission to final career recommendation—was tested to verify smooth data flow between the frontend, backend API, NLP module, ML classifier, and the Users table.

Testing confirmed that the system correctly processed form inputs, extracted keywords and sentiment, generated accurate predictions, and produced structured career roadmaps. Database entries were checked for proper storage and updates of user details and analysis results.

Accuracy and Usability were also assessed by simulating multiple user requests and testing edge cases such as incomplete forms or lengthy essays. Overall, system testing validated that BhabishyaBot operates reliably, handles errors gracefully, and delivers consistent end-to-end functionality.

Chapter 6: CONCLUSION AND FUTURE RECOMMENDATION

6.1 Conclusion

The BhabishyaBot project aimed to develop an AI-powered career guidance system that assists users in exploring personalized career paths based on their personal information, interests, and written essays. The project successfully implemented several core modules, including:

- A chatbot interface for sequential data collection and user interaction.
- Database integration using SQL Server to securely store user profiles and essays.

Unit testing of critical modules, including database operations, chatbot logic, and API endpoints, confirmed the reliability and correctness of individual components. Although the system is not fully deployed with a complete recommendation roadmap and advanced features, the current implementation demonstrates the feasibility of integrating AI-driven essay analysis with career guidance functionalities.

Overall, BhabishyaBot provides a structured framework for intelligent career counselling. It highlights how conversational AI and machine learning can assist in personalized guidance while maintaining modular and maintainable system architecture.

6.2 Future Recommendations

Several enhancements can be considered for the future development of BhabishyaBot:

1. Full System Deployment and End-to-End Testing:
6. Complete the integration of all modules and conduct comprehensive system testing, including frontend-backend interaction, real-time recommendation delivery, and user experience validation.
2. Enhanced Chatbot Interaction:
7. Add more interactive and adaptive dialogue, including clarifying questions and personalized prompts.
8. Integrate multi-turn conversation memory to track user progress over multiple sessions.
3. Mobile Accessibility and Cross-Platform Support:
 - Implement responsive design and mobile application support to increase accessibility.
 - Include multilingual support to cater to users across different regions in Nepal and beyond.
4. Security and Privacy Enhancements:
 - Encrypt user data and secure API endpoints.
 - Implement privacy-compliant data handling, especially for sensitive user information.

REFERENCES

- [1] Sakir Hossain Faruque, “*Unlocking Futures: A Natural Language Driven Career Prediction System for Computer Science and Software Engineering Students.*”
arXiv preprint: <https://arxiv.org/abs/2405.18139>
- [2] Naveen Mathews Renji, Balaji R Rao, Carlo Lipizzi, “*Steve: LLM Powered ChatBot for Career Progression.*”
arXiv preprint: <https://arxiv.org/abs/2504.03789>
- [3] Harshita, “*Career Buddy – Career Guidance AI Chatbot.*”
GitHub Repository: <https://github.com/Harshita-b/Career-Buddy---Career-Guidance-AI-Chatbot>
- [4] Nigel Guenole, Sheri Feinzig, “*The Business Case for AI in Talent Development.*”
IBM Smarter Workforce Institute, 2018.
- [5] Shilaskar et al., “*AI-Driven Career Advisor using RASA Chatbot and ML-based Recommendations.*”
Preprint available via IEEE/arXiv (2024).
- [6] R. Casuat et al, “*Employability Prediction using Machine Learning Algorithms: A Philippine Study.*”
ResearchGate, 2020.
- [7] Ofoegbu et al, “*Career Guidance System Using Decision Tree, Random Forest, and Naïve Bayes Algorithm*”
ResearchGate, 2025
- [8] Probst et al, “*Hyperparameters and Tuning Strategies for Random Forest*”
ResearchGate, 2018

Appendices



Fig 12: Landing Page

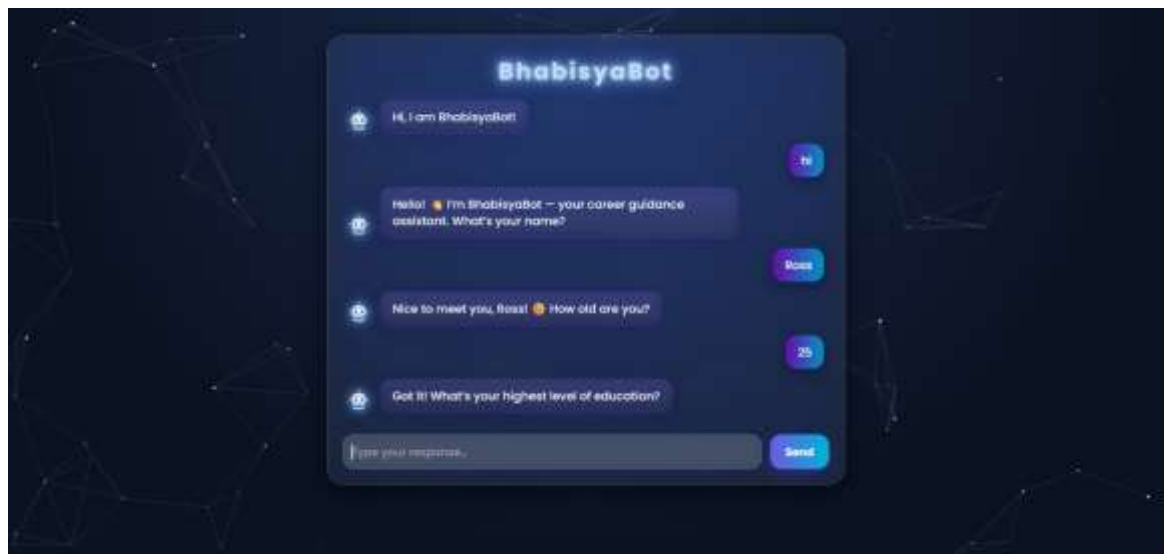


Fig 13: Chatbot Page

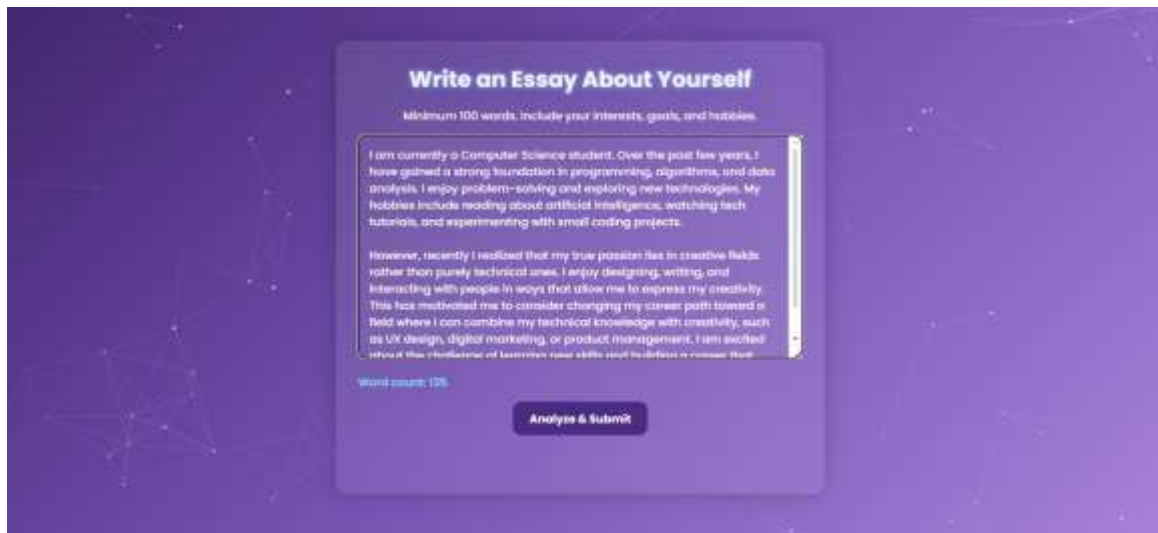


Fig 14: Essay Page

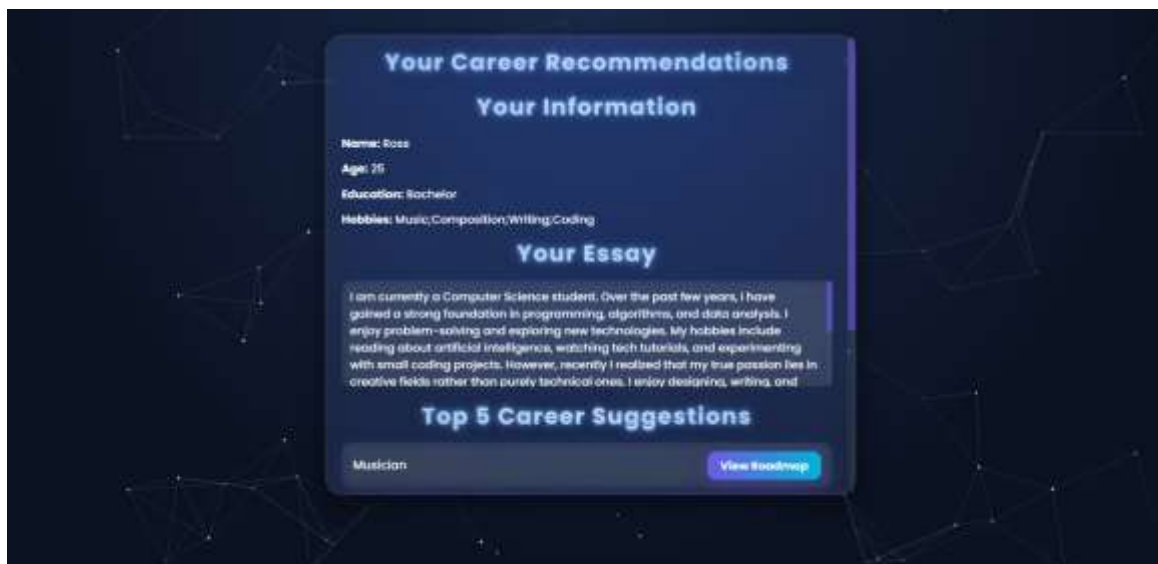


Fig 15: Career Recommendation Page1

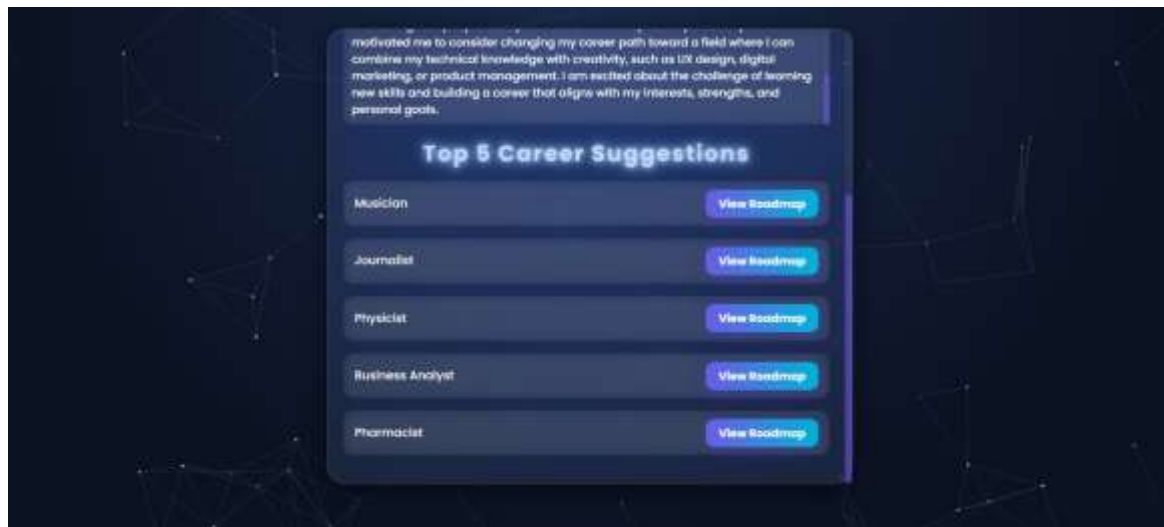


Fig 16: Career Recommendation Page2



Fig 17: Roadmap Page